Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación
IIC2223 - Teoría de Autómatas y Lenguajes Formales
Profesor: Cristian Riveros
Ayudante: Santiago Marcano

Ayudantía 10

Repaso I2

Problema 1

Considere el lenguaje

$$L = \{a^i b^j \mid i = j \text{ o } i = 2j\}.$$

Demuestre que L no es regular.

Solución Problema 1

Mostramos dos formas alternativas de demostrar que L no es regular.

Solución usando el Lema de Bombeo

Utilizaremos el contrapositivo del Lema de Bombeo.

Sea $N > 0$ arbitrario. Escogemos

$$x = \varepsilon, \qquad y = a^N, \qquad z = a^N b^N.$$

Entonces

$$xyz = a^{2N} b^N \in L,$$

pues la cantidad de a's es el doble de la cantidad de b's. Además, $|y| = N$.

Consideremos ahora una descomposición arbitraria $uvw = y$ con $v \neq \varepsilon$. Como $y = a^N$, existen $r, s, t \geq 0$, con $s > 0$, tales que

$$u = a^r, \qquad v = a^s, \qquad w = a^t,$$

y

$$r + s + t = N.$$

Escogemos $i = 2$. La palabra bombeada es

$$xuv^2wz = a^{2N+s} b^N.$$

Veamos que esta palabra no pertenece a L.

Por un lado, como $s > 0$,

$$2N + s > N,$$

por lo que

$$2N + s \neq N.$$

Por otro lado, nuevamente como $s > 0$,

$$2N + s > 2N,$$

por lo que

$$2N + s \neq 2N.$$

Así, la cantidad de a's no es igual a la cantidad de b's ni tampoco es el doble de la cantidad de b's. Por lo tanto,

$$xuv^2wz \notin L.$$

Como esto funciona para todo $N > 0$ y para toda descomposición $uvw = y$ con $v \neq \varepsilon$, por el contrapositivo del Lema de Bombeo concluimos que

$$\boxed{L \text{ no es regular.}}$$

Solución usando Myhill–Nerode

Para cada $n \geq 0$, tomamos la siguiente familia de palabras

$$S = \{a^{3^n} \mid n \geq 0\}$$

Mostraremos que estas palabras son distinguibles dos a dos respecto de L. Tomemos $x_i = a^{3^i}$ y $x_j = a^{3^j}$ donde $i, j \geq 0$, supongamos sin perdida de generalidad $i < j$. Ahora consideremos el sufijo

$$z = b^{3^i}.$$

Tenemos entonces lo siguiente:

- $x_i z = a^{3^i} b^{3^i} \in L$ pues ambos exponentes son iguales.
- $x_j z = a^{3^j} b^{3^i} \notin L$ ya que como $i < j$

  $$3^j \neq 3^i.$$

  y además

  $$3^j \neq 2 \cdot 3^i,$$

  pues esto implicaría

  $$3^{j-i} = 2,$$

  lo cual es imposible.

Así, para todo $i < j$, las palabras x_i y x_j son distinguibles respecto de L. Luego, cada palabra en S determina una clase de equivalencia distinta, y como cada clase esta representada por un número natural entonces $\equiv_L$ tiene infinitas clases de equivalencia.

Por el Teorema de Myhill–Nerode, concluimos que

$$\boxed{L \text{ no es regular.}}$$

Problema 2

Sea $T = (Q_T, \Sigma, \Omega, \Delta_T, I_T, F_T)$ un transductor no determinista sin transiciones ε de entrada, y sea $A = (Q_A, \Omega, \Delta_A, I_A, F_A)$ un NFA.

Diseñe un algoritmo on-the-fly que, dado $u \in \Sigma^*$, decida si

$$[\![T]\!](u) \cap L(A) \neq \emptyset.$$

Solución Problema 2

Sea $u = a_1 a_2 \cdots a_n$. Mantendremos un conjunto

$$S \subseteq Q_T \times Q_A,$$

donde $(p, q) \in S$ significa que existe una ejecución del transductor sobre el prefijo procesado que termina en p y produce una palabra v, y existe una ejecución del NFA sobre v que termina en q.

Algoritmo 1 T-NFA-INTERSECT(T, A, u)

```
1:  S ← I_T × I_A
2:  for i = 1 to n do
3:      S_old ← S
4:      S ← ∅
5:      for all (p, q) ∈ S_old do
6:          for all (p, a_i, b, p') ∈ Δ_T do
7:              if b = ε then
8:                  S ← S ∪ {(p', q)}
9:              else
10:                 for all q' tal que (q, b, q') ∈ Δ_A do
11:                     S ← S ∪ {(p', q')}
12:                 end for
13:             end if
14:         end for
15:     end for
16: end for
17: return check (S ∩ (F_T × F_A) ≠ ∅)
```

se cumple que

$$(p, q) \in S$$

si, y solo si, existe una ejecución de T desde algún estado de I_T hasta p, leyendo $a_1 \cdots a_i$ y produciendo alguna palabra v, y existe una ejecución de A desde algún estado de I_A hasta q leyendo v.

Caso base.

Antes de leer símbolos de u, tenemos

$$S = I_T \times I_A.$$

El transductor no ha leído ningún símbolo y ha producido ε. Asimismo, el NFA no ha leído ningún símbolo. Por lo tanto, el invariante se cumple.

Paso inductivo.

Supongamos que el invariante se cumple después de procesar $a_1 \cdots a_{i-1}$.

Considere un par

$$(p, q) \in S_{\text{old}}.$$

Por hipótesis inductiva, existe una ejecución del transductor que termina en p y produce alguna palabra v, y una ejecución de A que termina en q después de leer v.

Si existe una transición

$$(p, a_i, x, p') \in \Delta_T,$$

podemos extender la ejecución de T leyendo a_i. La nueva salida es

$$vx.$$

Para cada q' tal que

$$(q, x, q') \in \Delta_A$$

podemos extender también la ejecución de A desde q hasta q' leyendo x.

Por lo tanto, el algoritmo agrega

$$(p', q')$$

a S.

Recíprocamente, todo par agregado a S se obtiene de esta forma a partir de un par de S_{old}, una transición válida del transductor y una ejecución válida del NFA sobre la salida de dicha transición.

Luego, el invariante se mantiene.

Correctitud

Después de procesar toda la palabra u, el algoritmo retorna TRUE si existe

$$(p, q) \in S$$

tal que

$$p \in F_T \qquad \text{y} \qquad q \in F_A.$$

Por el invariante, esto ocurre si, y solo si, existe una ejecución aceptante de T sobre u que produce una palabra v aceptada por A.

Es decir,

$$v \in [\![T]\!](u) \qquad \text{y} \qquad v \in L(A).$$

Por lo tanto,

$$[\![T]\!](u) \cap L(A) \neq \emptyset.$$

Concluimos que el algoritmo es correcto.

Problema 3

Sea $G = (V, \Sigma, P, S)$ una gramática libre de contexto en forma normal de Chomsky, con $\Sigma = \{a, b\}$

Construya una gramática libre de contexto

$$G' = (V', \Sigma, P', S')$$

tal que

$$L(G') = \{w \in L(G) \mid \#_a(w) - \#_b(w) \equiv 0 \pmod 3\}.$$

La gramática construida debe estar también en forma normal de Chomsky.

Solución Problema 3

La idea es guardar, junto a cada variable de la gramática original, el residuo módulo 3 de

$$\#_a(w) - \#_b(w)$$

de la palabra generada.

Para cada variable $X \in V$, creamos tres variables

$$X_0, \qquad X_1, \qquad X_2.$$

La interpretación será que

$$X_r \Rightarrow_{G'}^{*} w$$

si, y solo si,

$$X \Rightarrow_{G}^{*} w$$

y

$$\#_a(w) - \#_b(w) \equiv r \pmod 3.$$

Definimos

$$V' = \{X_r \mid X \in V,\ r \in \{0,1,2\}\}.$$

Como queremos quedarnos con las palabras cuyo residuo es 0, tomamos como símbolo inicial

$$S' = S_0.$$

Ahora construimos las producciones de G'.

Producciones terminales

Como G está en forma normal de Chomsky, una producción terminal es de la forma

$$X \to a \text{ ó } X \to b.$$

Si

$$X \to a,$$

entonces

$$\#_a(a) - \#_b(a) = 1,$$

por lo que agregamos

$$X_1 \to a.$$

Si

$$X \to b,$$

entonces

$$\#_a(b) - \#_b(b) = -1 \equiv 2 \pmod 3,$$

por lo que agregamos

$$X_2 \to b.$$

Producciones binarias

Consideremos ahora una producción de G de la forma

$$X \to YZ.$$

Supongamos que Y genera una palabra u tal que

$$\#_a(u) - \#_b(u) \equiv r \pmod 3,$$

y que Z genera una palabra v tal que

$$\#_a(v) - \#_b(v) \equiv s \pmod 3.$$

Como X genera la concatenación uv, tenemos

$$\begin{aligned} \#_a(uv) - \#_b(uv) &= \#_a(u) + \#_a(v) - \#_b(u) - \#_b(v) \\ &= (\#_a(u) - \#_b(u)) + (\#_a(v) - \#_b(v)) \,. \end{aligned}$$

Por lo tanto,

$$\#_a(uv) - \#_b(uv) \equiv r + s \pmod 3.$$

Así, por cada producción

$$X \to YZ$$

y para todo

$$r, s \in \{0, 1, 2\},$$

agregamos la producción

$$X_{(r+s) \bmod 3} \to Y_r Z_s.$$

Equivalentemente, las nueve combinaciones son

$$\begin{aligned} X_0 &\to Y_0 Z_0, \\ X_1 &\to Y_0 Z_1, \\ X_2 &\to Y_0 Z_2, \\ X_1 &\to Y_1 Z_0, \\ X_2 &\to Y_1 Z_1, \\ X_0 &\to Y_1 Z_2, \\ X_2 &\to Y_2 Z_0, \\ X_0 &\to Y_2 Z_1, \\ X_1 &\to Y_2 Z_2. \end{aligned}$$

Definimos P' como el conjunto de todas las producciones obtenidas a partir de las producciones de P mediante las construcciones anteriores.

Por lo tanto,

$$G' = (V', \{a, b\}, P', S_0).$$

Propuesto

Considere la relación

$$R_{\text{del}} = \{(u, v) \in \{a, b\}^* \times \{a, b\}^* \mid v \text{ se obtiene eliminando exactamente un símbolo de } u\}.$$

Construya un transductor no determinista T_{del} tal que $[\![T_{\text{del}}]\!] = R_{\text{del}}$.

Solución Propuesto

Construimos un transductor con dos estados:

$$Q = \{q_0, q_1\}.$$

La interpretación de los estados será:

$$q_0 : \text{ todavía no se ha eliminado ningún símbolo,}$$
$$q_1 : \text{ ya se eliminó exactamente un símbolo.}$$

Definimos

$$T_{\text{del}} = (Q, \{a, b\}, \{a, b\}, \Delta, \{q_0\}, \{q_1\}).$$

Para cada

$$x \in \{a, b\},$$

agregamos las siguientes transiciones:

$$(q_0, x, x, q_0),$$
$$(q_0, x, \varepsilon, q_1),$$
$$(q_1, x, x, q_1).$$

Las transiciones

$$(q_0, x, x, q_0)$$

permiten copiar símbolos antes de realizar la eliminación.

La transición

$$(q_0, x, \varepsilon, q_1)$$

consume un símbolo de la entrada sin producir nada, por lo que elimina exactamente ese símbolo.

Finalmente,

$$(q_1, x, x, q_1)$$

copia todos los símbolos restantes.

Como q_0 no es final, toda ejecución aceptante debe realizar al menos una eliminación. Como desde q_1 no existe ninguna transición que produzca ε, una ejecución aceptante no puede eliminar más de un símbolo.

Por lo tanto,

$$\boxed{[\![T_{\text{del}}]\!] = R_{\text{del}}.}$$